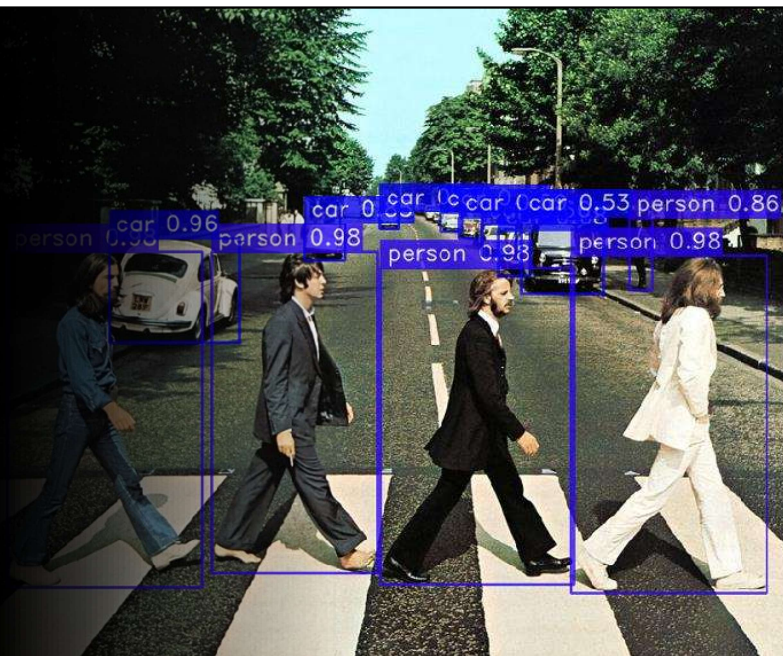


Efficient polyp detection using YOLOv12

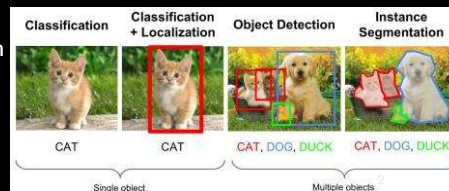
Ana T. Sanchez
Advisor: Dr. Pengfei Gu
05/12/2025



Hello everyone. Today I'll be presenting my project titled *Efficient Polyp Detection*. I used YOLOv12m to detect polyps from colonoscopy images, with the goal of having the model both accurate and computationally efficient.

General Problem of AI Detection

- What is AI detection?
 - refers to using artificial intelligence, (especially deep learning models), to identify and locate objects in data such as images, videos, or audio
- Classification and localization
 - Model analyzes the input
 - Identifies potential object locations
 - Draws bounding boxes around the object(s)
 - Assigns a label to each detected object
- General Challenges:
 - Data dependency: requires large labeled datasets to train effectively
 - Overfitting risk: models may memorize training images instead of generalizing
 - Small object detection: hard to detect small or irregular objects due to low resolution



Ana T. Sanchez

AI detection uses deep learning to identify and locate objects in images, video, or audio. The model analyzes the input, identifies potential object locations, draws bounding boxes, and assigns a label to each one. Common challenges include needing large labeled datasets, the risk of overfitting to training data, and difficulty detecting small or irregular objects.

Our problem

- Objective:
 - Efficiently detect polyps in colonoscopy images using a lightweight AI model
- Challenges:
 - Irregular polyp shapes and textures
 - High variability in color and size

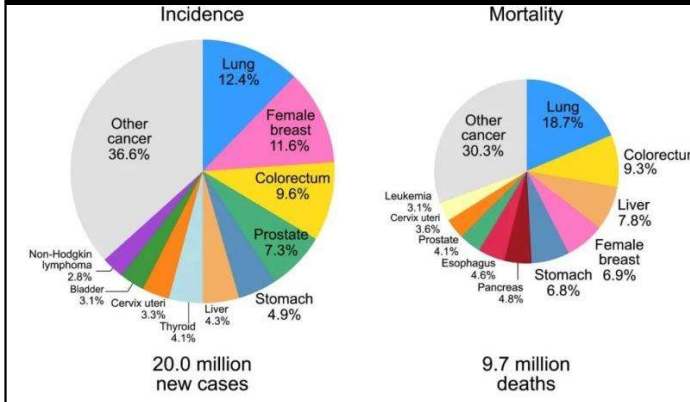


Ana T. Sanchez

3

The goal was to efficiently detect polyps in colonoscopy images using a lightweight model. This is a difficult task because polyps vary a lot in shape, texture, color, and size. I chose YOLOv12 because it's optimized for speed and performance, with newer features suited to hard-to-detect targets like polyps.

Why polyps?



- Polyps are early indicators of colorectal cancer (CRC).
- CRC is the 3rd most common cancer worldwide.
- Early detection significantly improves treatment outcomes.
- Miss rate for polyps can be as high as 22% during traditional screening.
- Automation can reduce human error and workload in clinical settings.

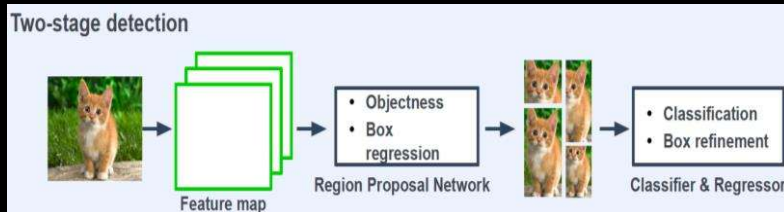
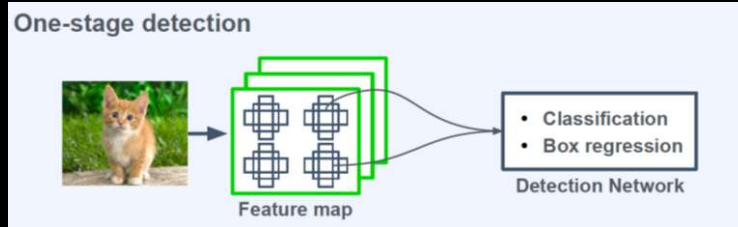
Ana T. Sanchez

4

Polyps are early indicators of colorectal cancer, which is the third most common cancer worldwide. Catching them early significantly improves treatment outcomes, but the miss rate during traditional screening can be as high as 22%. Automated detection can help reduce human error and workload in clinical settings.

Background/Related Work

- One-stage detectors:
 - SSD, YOLO(v1-v12)
 - How it works:
 - Predict bounding boxes and classes in one pass
 - Pros: fast, lightweight
 - Cons: earlier versions are less accurate

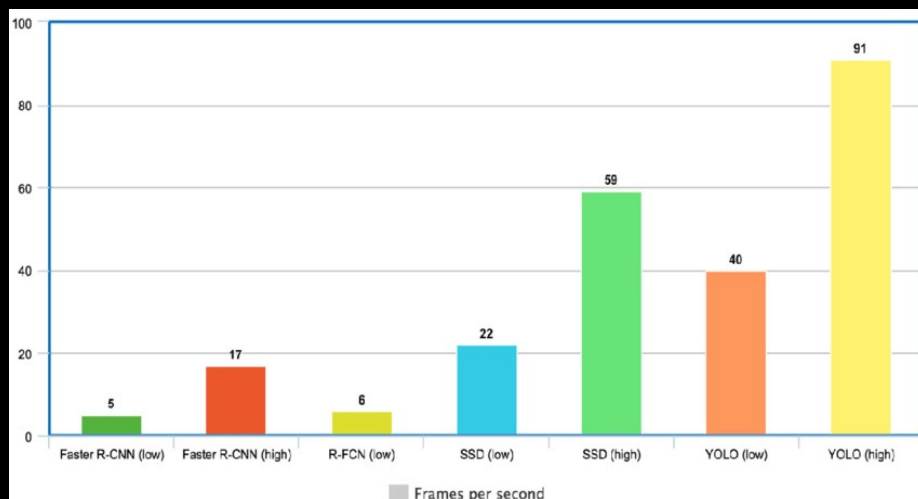


Ana T. Sanchez

- Two-stage detectors
 - R-CNN, Faster R-CNN
 - How it works:
 - First, generate region proposals, then classify each
 - Pros: Very accurate
 - Cons: Slow, heavy unsuitable for real-time use

5

Object detectors generally fall into two categories. One-stage detectors, like SSD and YOLO, predict bounding boxes and classes in a single pass, making them fast and lightweight, though earlier versions were less accurate. Two-stage detectors, like R-CNN and Faster R-CNN, first generate region proposals and then classify each one. They're more accurate but slower and less suited for real-time use.



Frames per second (FPS) is a metric that lets us compare the speed of different object detectors.

SSD and YOLO are one-stage object detectors.

Faster-RCNN and R-FCN are two-stage object detectors

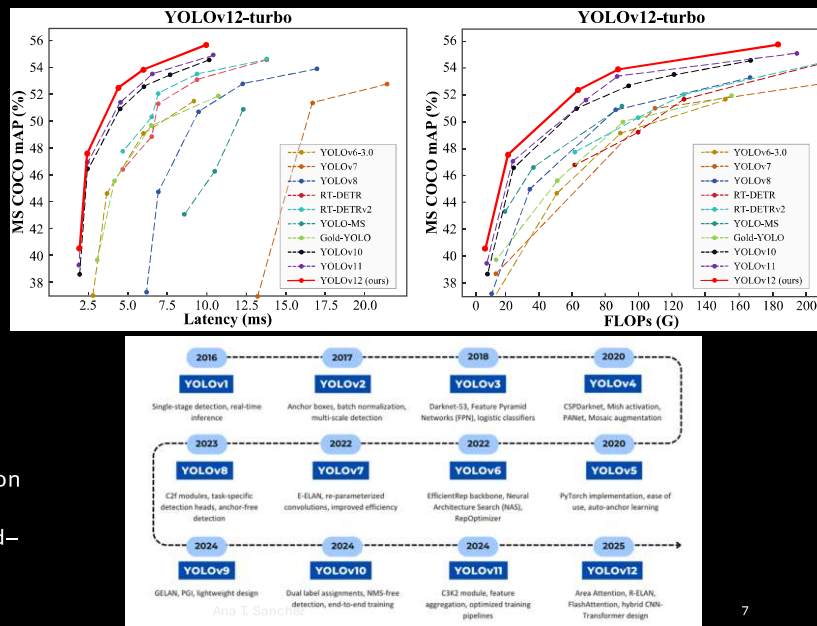
Ana T. Sanchez

6

Frames per second is a common way to compare detector speed. This chart shows that one-stage detectors like SSD and YOLO run considerably faster than two-stage detectors like Faster R-CNN and R-FCN.

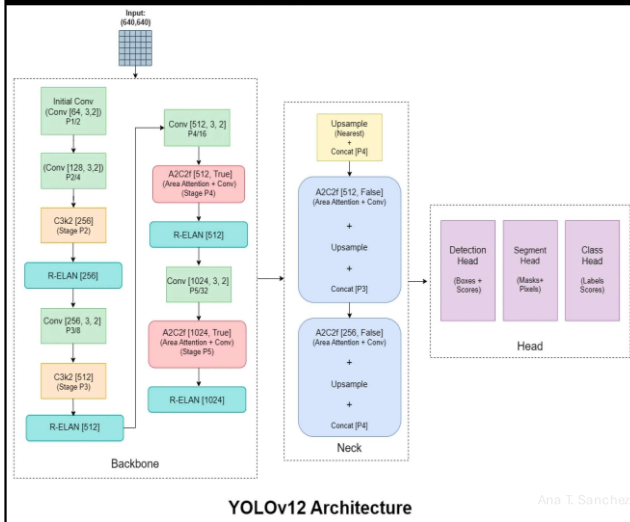
YOLO Evolution

- YOLOv1–v4: classic CNN-based architecture.
- YOLOv5–v8: improved anchor box handling, augmentation, loss functions.
- YOLOv12:
 - Adds better attention mechanisms.
 - Uses spatial information more effectively.
 - Aims for optimal speed–accuracy balance.



YOLOv1 through v4 used a classic CNN-based architecture. Versions 5 through 8 improved anchor box handling, data augmentation, and loss functions. YOLOv12 adds better attention mechanisms, makes more effective use of spatial information, and aims for an optimal balance between speed and accuracy.

Method Overview - YOLOv12 Architecture



Backbone: Extracts spatial features from the input image.

Neck (FPN + PAN): Fuses features at different scales (important for small objects).

Head: Directly predicts bounding box coordinates, class confidence.

Core Concepts:

Anchor-free detection: Predicts bounding boxes directly without pre-defined anchors (reduces computation).

Loss Function: Combines classification loss, objectness loss, and IoU-based localization loss (CIoU/DIoU).

Innovations:

“Area Attention”

“R-ELAN”

Enhanced loss function (e.g., CIoU/DIoU + Focal Loss).

The architecture has three main parts.

The backbone extracts spatial features from the input image.

The neck, using FPN and PAN, fuses features across different scales, which matters for detecting small objects.

The head directly predicts bounding box coordinates and class confidence.

YOLOv12 is anchor-free, meaning it predicts boxes without relying on predefined anchor shapes, which reduces computation.

Its loss function combines classification loss, objectness loss, and IoU-based localization loss (CIoU/DIoU).

Key innovations include Area Attention and R-ELAN. This modular design makes it easy to upgrade individual parts, like adding attention in the neck or swapping in a different loss function.

R-ELAN improves on the earlier ELAN architecture with block-level residual connections and scaling techniques for stable training, along with a redesigned feature aggregation method that improves both performance and efficiency.

Area Attention Module

- Divides feature maps into equal-sized regions (either vertically or horizontally)
- This significantly reduces computational cost compared to standard self-attention



The Area Attention Module divides the feature map into equal sized segments, which significantly reduces computational cost compared to standard self-attention while still preserving a large receptive field.

This simple modification allows the model to retain a significant field of view while improving speed and efficiency.

Two architectural changes stand out:

Area Attention module: Provides much faster inference with minimal accuracy loss.

R-ELAN module: Ensures that even very deep or large-atthelps ensure that even deep or large attention-based models converge reliably.

Experimental Setup

- Model: YOLOv12
- Epochs: 300
- Early Stopping Patience: 0
- Optimizer: AdamW
- Batch Size: 256
- Hardware: NVIDIA A100 GPU
- Frameworks: PyTorch + Weights & Biases (wandb)

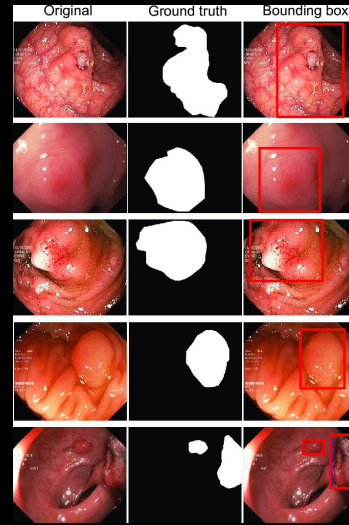
Ana T. Sanchez

10

YOLOv12m was used for this project. Trained for up to 300 epochs with no early stopping. All training was done on an NVIDIA A100 GPU with logging and experiment tracking through "Weights & Biases."

Dataset Description - Kvasir-SEG

- Domain: Medical imaging — gastrointestinal endoscopy
- Source: Collected at Vestre Viken Health Trust, Norway
- Published by: Simula Research Laboratory
- Data Format:
 - 1000+ polyp images
 - Corresponding segmentation masks (1 per image)
- Training images: 800
- Validation/Test images: 100
- Resolution: Varies (resized 640px)
- Classes: Single-class — "polyp"
- Label Format for YOLO:
 - Segmentation masks converted to bounding boxes
 - Saved in .txt files as YOLO format (class, x_center, y_center, width, height)



Ana T. Sanchez

11

I used the Kvasir-SEG dataset, collected at Vestre Viken Health Trust in Norway and published by Simula Research Laboratory.

The Kvasir-SEG dataset contains 1000 polyp images and their corresponding ground truth from the Kvasir Dataset v2.

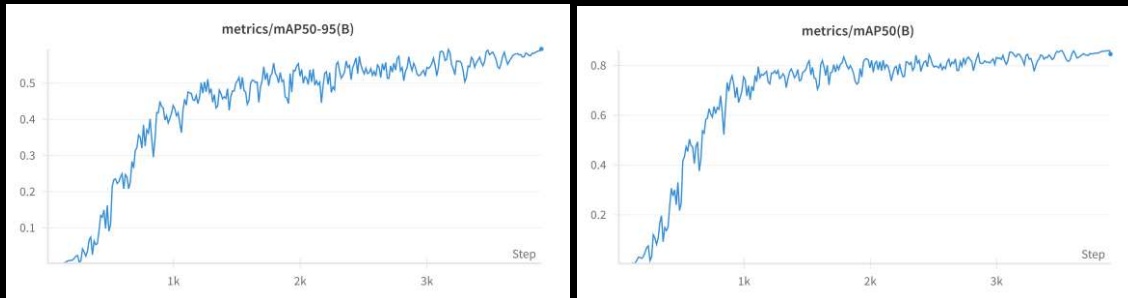
The resolution of the images contained in Kvasir-SEG varies from 332x487 to 1920x1072 pixels. Images were resized to 640 pixels.

The images and its corresponding masks are stored in two separate folders with the same filename.

The image files are encoded using JPEG compression, and online browsing is facilitated.

Since this is a single-class problem, I converted the segmentation masks into bounding boxes and saved labels in YOLO format (class, x_center, y_center, width, height).

Results- Quantitative



	mAP50	mAP50-95
KVASIR-SEG	0.86	0.591

Ana T. Sanchez

12

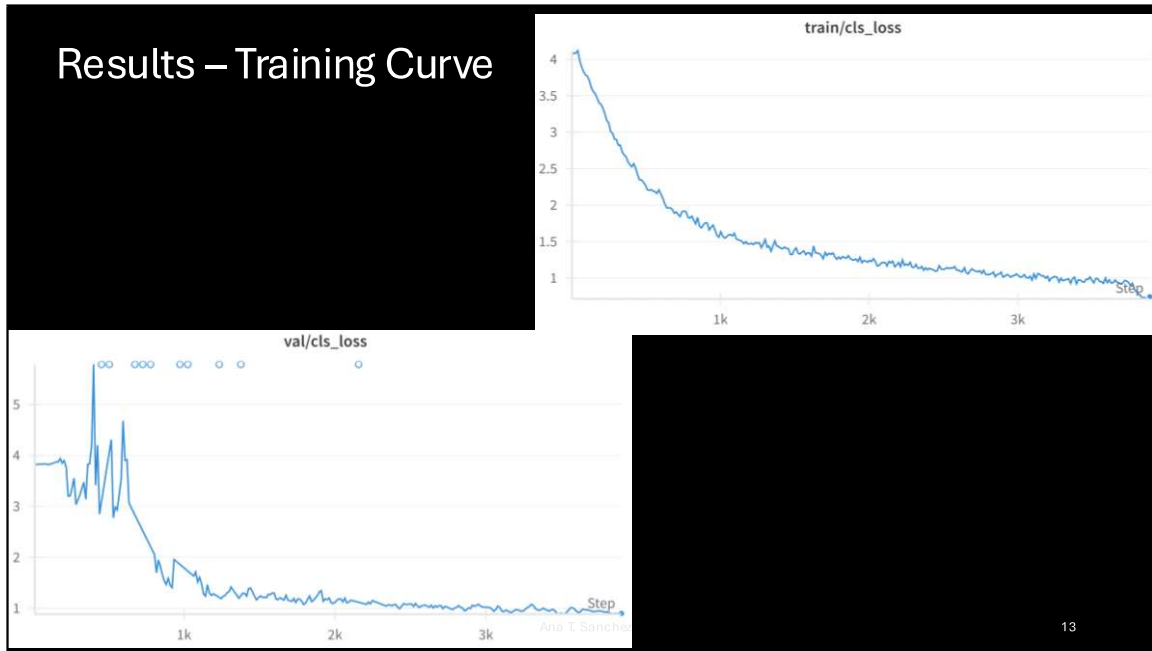
The left graph shows mAP across IoU thresholds from 0.5 to 0.95, in steps of 0.05, which is a stricter, more comprehensive metric. There's a clear upward trend that flattens around 0.55, suggesting the model is nearing convergence.

The right graph shows mAP at the more lenient IoU threshold of 0.5, which plateaus around 0.85 to 0.86. Overall, the model reached an mAP50 of 0.86 and an mAP50-95 of 0.591. The mAP50-95 score is lower because it's evaluated against much tighter IoU thresholds, but for a medical imaging task like this, these results reflect solid precision and recall.

In simpler terms:

This slide shows how well the model does at finding polyps correctly. Using a stricter test, it scored 0.591 out of 1. Using a more relaxed test, it scored 0.86 out of 1. Both numbers trending upward over training means the model kept getting better and better at its job the longer it trained.

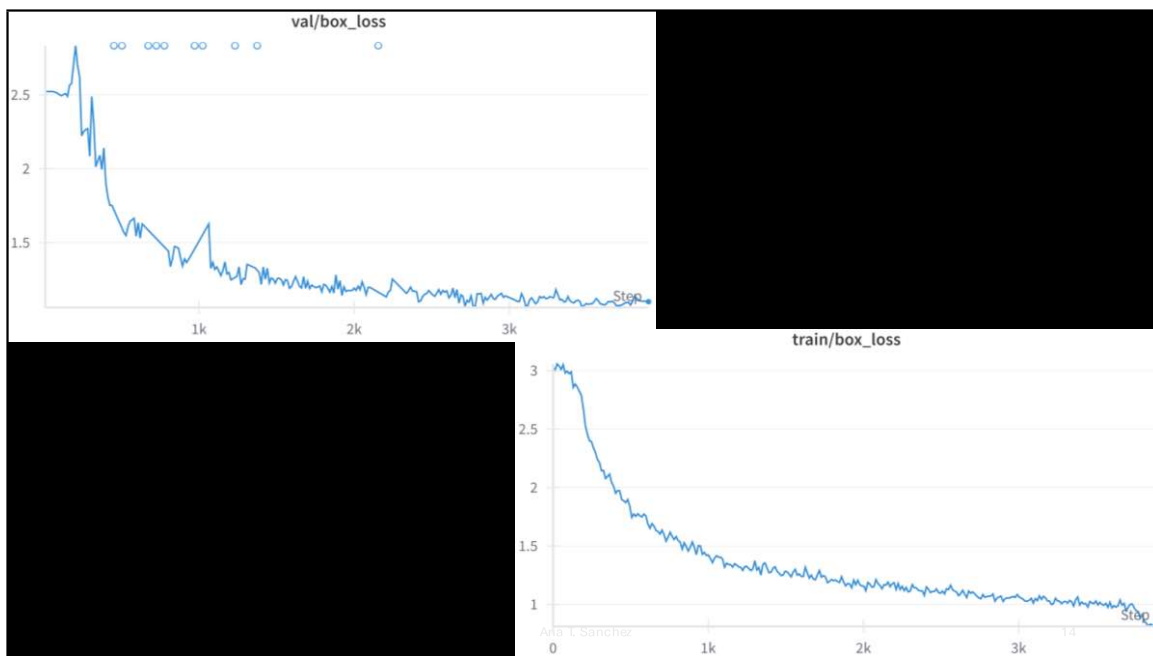
Results – Training Curve



The left chart shows classification loss on the training set, which starts around 4.1 and steadily decreases to about 0.9, showing the model is learning to classify the training data well.

The right chart shows classification loss on the validation set. It starts high with some noisy spikes early on, which is normal, but follows a downward trend and settles around 1.0 to 1.2. Since loss decreases on both training and validation sets, the model is learning without overfitting.

In simpler terms: "loss" is basically a measure of how wrong the model is, so lower is better. This slide shows the model's mistakes dropping steadily during training, both on the data it trained on and on new data it hadn't seen before. That's a good sign; it means the model was actually learning the pattern of what a polyp looks like, not just memorizing the training images.

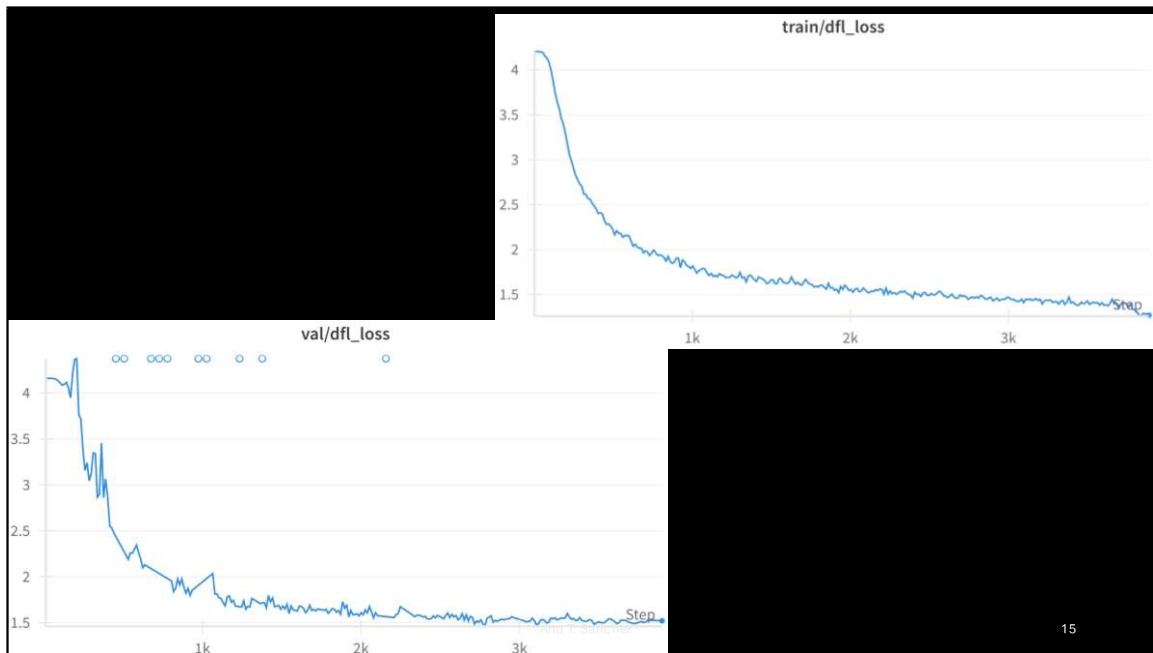


The left chart shows box loss on the validation set, starting above 2.5 and settling around 1.2 after some early fluctuation.

The right chart shows box loss on the training set, starting around 3.0 and steadily dropping below 1.0. This consistent decline on both sets shows the model is improving at predicting object locations and generalizing well to unseen data.

In simpler terms:

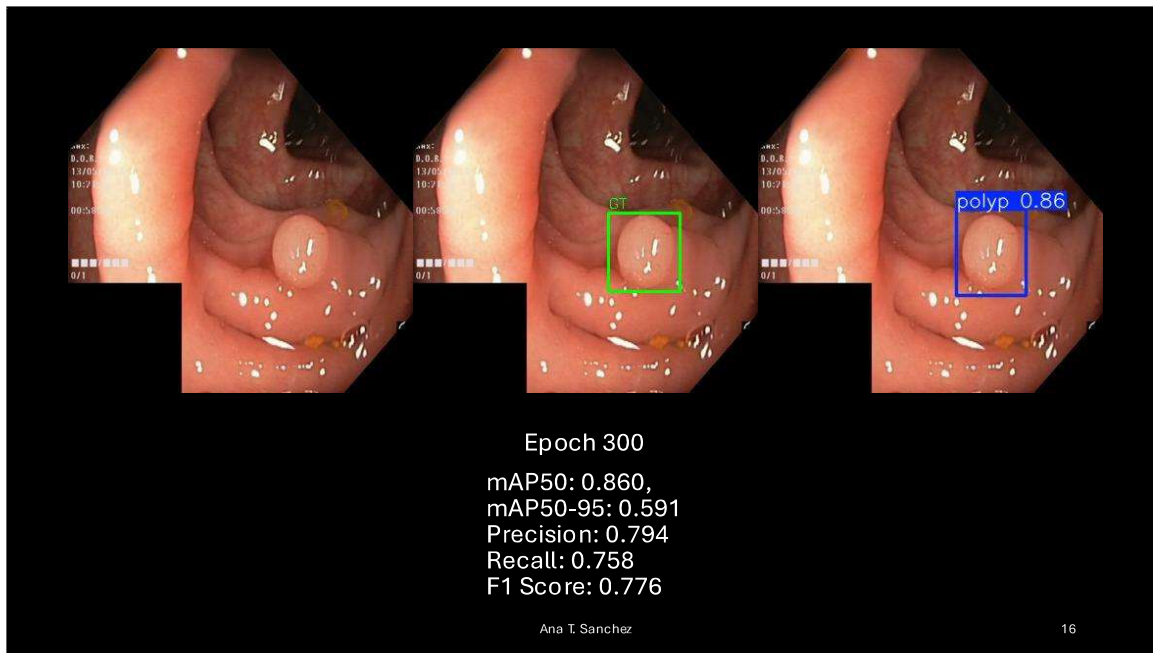
This measures how accurate the model's boxes were at framing the polyp, not just whether it found one, but how well it drew the box around it. There are really two separate skills here: noticing that a polyp is there at all, and then drawing a tight, accurate box around it. A model could technically get the first part right while still drawing a sloppy box, too big, too small, or off to the side. Since this box error dropped steadily throughout training, it shows the model wasn't just getting better at spotting polyps, it was also getting better at framing them accurately, which matters if this were ever used to help point someone toward exactly where to look.



Both training and validation DFL loss start around 4.1 and decrease over time. Training loss drops smoothly, while validation loss drops with some early instability before settling, which indicates the model is learning to generalize rather than just memorizing the training data. Across classification, box, and DFL loss, the consistent downward trend suggests the model is converging well without overfitting.

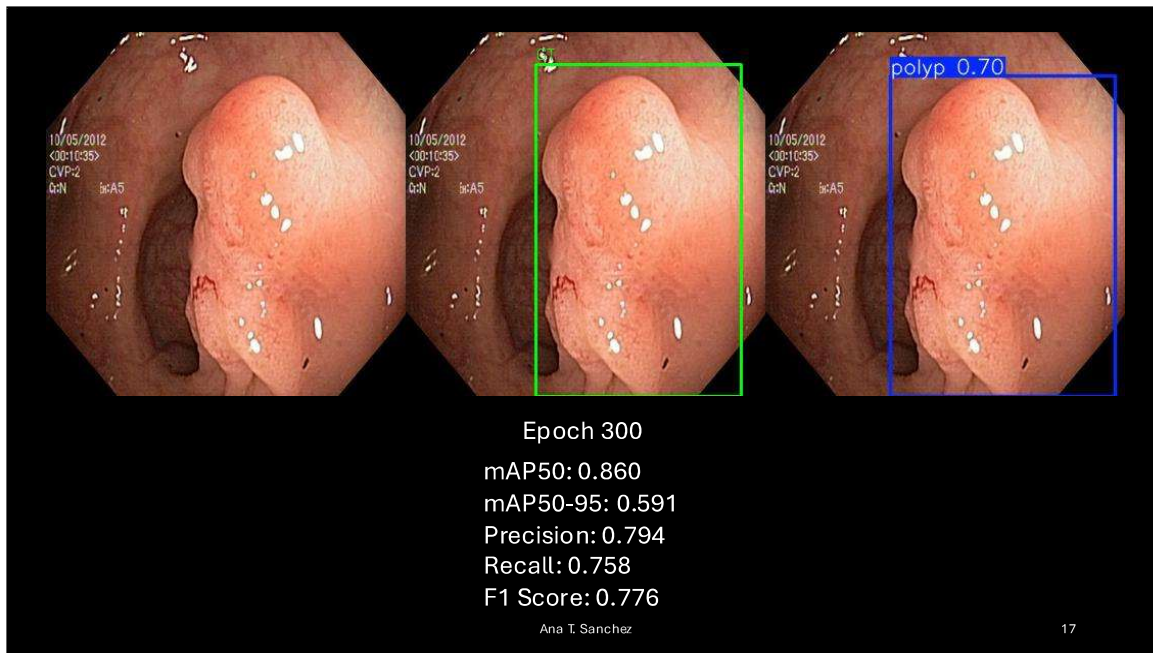
In simple terms:

this is another measure related to how precisely the model draws its boxes. It also dropped over time, with a bit of a bumpy start, which is normal early in training. The bumps smoothing out over time is a good sign that the model was becoming more stable and reliable.

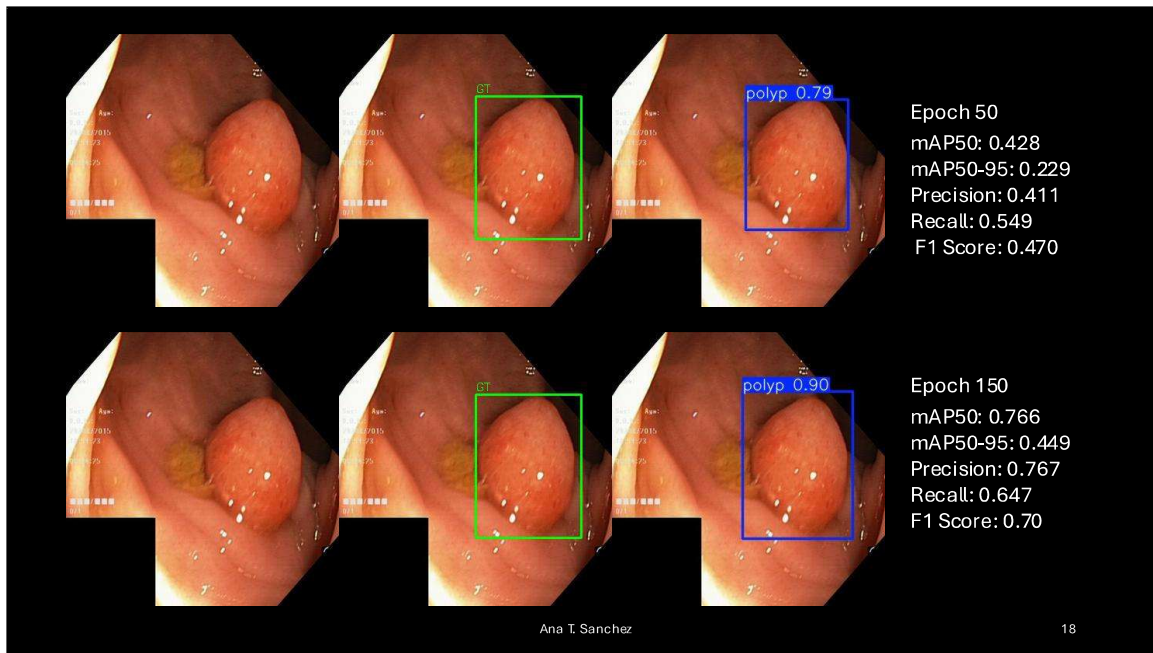


Here you can see some visual outputs.

These images show how the model performs. The green boxes is the ground truth. The blue box is what the model detected with its confidence level



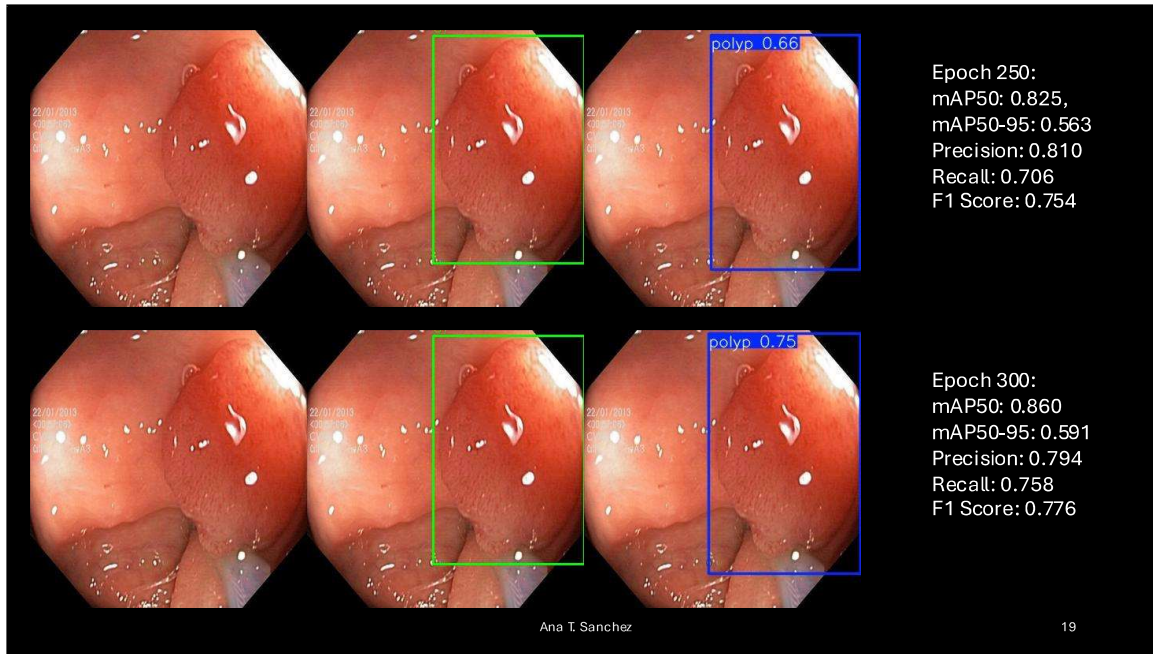
Another example of ground truth versus the model's prediction, with the confidence score shown for the detected polyp.



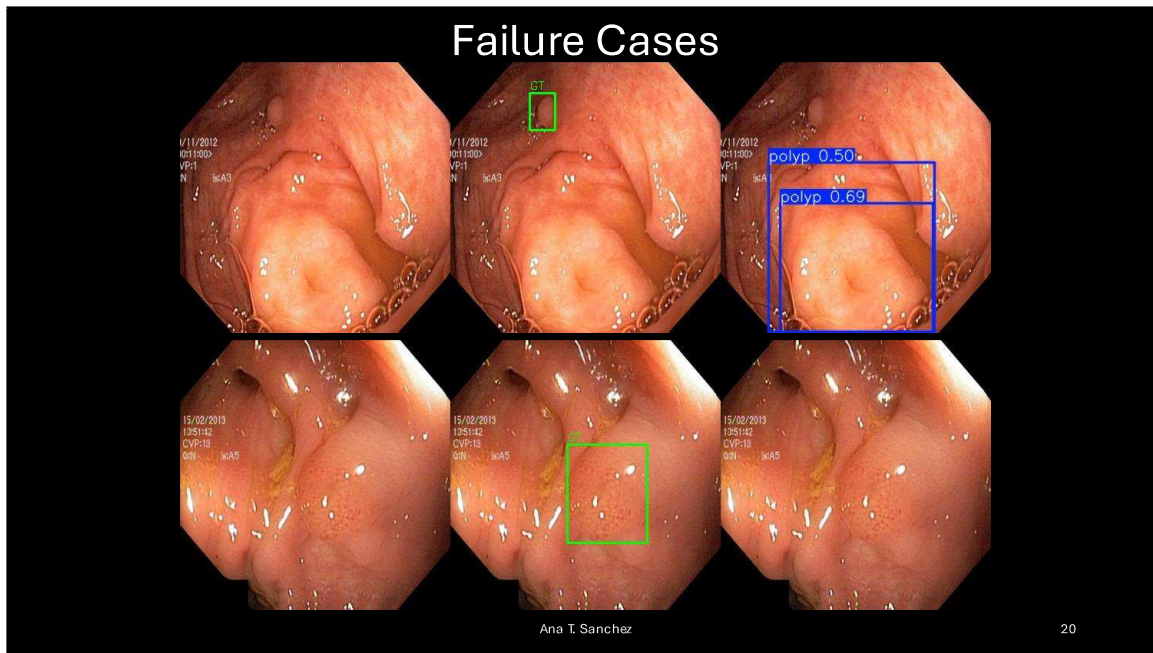
These images shows how detection quality improved over training.

At epoch 50, mAP50 was 0.428.

By epoch 150, it had improved to 0.766, with better box alignment and higher confidence in the detection



Failure Cases

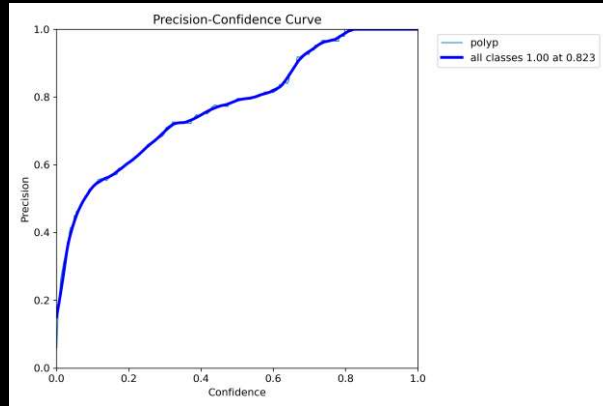


In the first example, the model misclassified part of the colon wall as a polyp, while the actual polyp was small and distant.

In the second example, the model missed the polyp entirely, likely due to strong reflection and glare from the endoscopic lighting, which reduces contrast between the polyp and surrounding tissue and makes it harder for the model to pick up distinguishing features.

Observations

- Strengths:
 - Detects small, low-contrast polyps.
 - Fast inference time: ~8ms per image.
- Weaknesses:
 - Some false positives and negatives due to texture confusion.
 - Still struggles with blurred edges, poor lighting or too much lighting
- Lessons:
 - Attention modules helped improve accuracy on ambiguous polyps.
 - Data augmentation significantly improved generalization.



Ana T. Sanchez

21

Strengths include detecting small, low-contrast polyps and fast inference at around 8ms per image.

Weaknesses include some false positives and negatives from texture confusion, and continued struggles with blurred edges or difficult lighting, whether too dim or too bright.

Key takeaways: attention modules helped improve accuracy on ambiguous polyps, and data augmentation meaningfully improved generalization.

Conclusion

- YOLOv12 proves to be effective and efficient for polyp detection.
- Achieved high accuracy with fewer resources.
- Impact:
 - Potential deployment in real-time screening tools.
 - Reduces error in manual examination.

Future work

- **Model:**
 - Improve detection under challenging conditions, including glare, low contrast, and distant/dim polyps.
 - Explore glare-resistant preprocessing methods or training on gloss-augmented samples
- **Dataset Expansion:**
 - Include more public datasets (CVC-ClinicDB, EndoScene, ETIS-Larib, Hyper-Kvasir).
- **Real-time Deployment:**
 - Deploy on Jetson Nano or Raspberry Pi to test in real-time embedded scenarios.

References

- Romero, H & Morales Acosta, Alex. (2020). A review: Comparison of performance metrics of pretrained models for object detection using the TensorFlow framework. IOP Conference Series: Materials Science and Engineering. 844. 012024. 10.1088/1757-899X/844/1/012024.
- [Colorectum | American Cancer Society](#)
- <https://www.who.int/news-room/fact-sheets/detail/colorectal-cancer>
- https://dl.acm.org/doi/10.1007/978-3-030-37734-2_37
- <https://datasets.simula.no/kvasir-seg/>

- <https://ieeexplore.ieee.org/abstract/document/10766572>
- <https://my.clevelandclinic.org/health/diseases/15370-colon-polyps>
- https://ieeexplore.ieee.org/abstract/document/10380749?casa_token=n93Rs4Xlr4AAAAAA:wgQmxmdunuzxnRj4FWFe-TdiQLL-UO2eNJPSTg99Lyj0qzWLIOWWebPd9GkPJubJMST96NEXop4
- https://ieeexplore.ieee.org/abstract/document/10671291?casa_token=lMeVqOyA7mIAAAAA:683VfmDPUIIMAKTDXbnmuzTvha6esO0fqWz-rPQedScdjmdiaAccZ-Ea73vS3AEHV7oLFLKVh2j3g

Thank you!!!

Ana T. Sanchez

26

Thank you to my advisor, Dr. Gu, for his time, patience, and guidance throughout this project.

I'd also like to thank the Computer Science faculty for allowing me to use the cluster.

Thank you all for your time, I hope you found this presentation informative.

Q&A -Detection Method Choice

Why YOLOv12m? Why not Mask R-CNN or DeepLabV3?

- YOLOv12 balances speed and accuracy, critical for real-time use.
- One-stage detection is better suited for real-time applications.
- Mask R-CNN is heavier and slower.

Ana T. Sanchez

27

I chose YOLOv12m because it's optimized for both speed and accuracy. Compared to earlier YOLO versions, it includes architectural improvements like better feature fusion and a streamlined head. It's faster than YOLOv8 and smaller than YOLOv5-large, which makes it perfect for real-time deployment on resource-constrained devices.

Q&A - Model Convergence

How do you know it converged?

- Training loss plateaued
- Validation mAP stabilized
- Loss curve stabilized after 230–250 epochs.
- Validation accuracy (mAP) plateaued — further training showed no gain

Q&A – Data Augmentation

Any data augmentation used?

- Yes!
 - Mosaic (1.0): Combines 4 training images into one. This boosts the model's ability to detect objects in varied contexts and scales by enriching image diversity.
 - Copy-Paste (0.1): With a probability of 10%, objects (e.g., polyps) are copied from one image and pasted into another. This helps increase the number of training examples with polyps.
 - Scale (0.5): Images are randomly scaled by up to $\pm 50\%$. This teaches the model to recognize polyps at different sizes or distances.

Ana T. Sanchez

29

Key data augmentation techniques applied during the YOLOv12 training process to improve the model's robustness and generalization capabilities.

Mosaic Augmentation:

The first technique is mosaic augmentation, which combines multiple images into one. This helps to expose the model to more diverse object appearances and scenarios. By stitching images together, we create a variety of contexts, helping the model recognize objects in different settings. In our case, this is applied with a mosaic value of 1.0 to ensure it's used during training.

Copy-Paste:

This randomly pastes objects from one image onto another, which helps the model become more invariant to object positioning and occlusion. It's particularly useful when training with limited data, as it artificially increases the diversity of scenes. In this setup, copy-paste is applied with a value of 0.1, meaning 10% of the training images undergo this augmentation.

By using these techniques, we help the model learn to generalize better and perform well on unseen data, despite variations in object appearances and placements.